

Welcome to 151B: Deep Learning

Instructor: Annapurna Vadaparty

TA: Ben Bao

Tutors: Alexander Radulescu, Trevor Duong

Some slides from Prof. Gary Cottrell “Transformer Networks”

Grades & Deadlines

- To receive your A1 programming grade: make sure to fill out the PL question
- Please submit your project proposal updates if you haven't already. If you need more time, please make a private Piazza post to let me know when this week you are planning to submit it. You will still be expected to meet your goal for the Week 4 checkpoint you set up for yourself.

Transformer Networks (NeurIPS 2017)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

Who has heard of this paper?

Who knew that it was a reference to the Beatles song “Love is All You Need”?

We've talked about representing time

- Two approaches:
 - Map time into space – we previously said this is tricky because it only works well with a small, fixed number of inputs
 - Map time into the state of the network (RNNs/LSTMs)
- Let's revisit mapping time into space—what if you had a model that only processes (*attends to*) a fixed number of inputs at a time, but then was able to share information about those

What is Attention?

- When you focus on a particular aspect of your environment - “pointing” to something
- *Overt* attention is when you move your eyes towards something or someone
- *Covert* attention is when you direct your attention to something without moving your eyes – it can even be attending to something in your thoughts!
- How can we “point” with neural networks?

Attention in Neural Networks

- How can we “point” with neural networks?
- We can control, e.g., camera movements (overt attention)
- We can direct attention to *internal* activations (covert attention)

We'll learn more about the process of attention tomorrow, but in order to know how this process works, it'll be easier if know what parts of what data we're attending to.

How we'll learn about modern transformers

1. What is our data and why? **Why language**
2. How are we representing the data? **Tokens and embeddings**
3. What is the architecture of the model we're using? **Transformers**
4. How does this architecture work? **Attention**

How we'll learn about modern transformers

What you should take away from today's class:

1. What is our data and why? **Why language**
2. How are we representing the data? **Tokens and embeddings**

What you can definitely learn from the pre-lecture content and other online materials:

3. What is the architecture of the model we're using? **Transformers**
4. How does this architecture work? **Attention**

Why focus so much on the data source?

- You're all very capable of understanding and recreating existing solutions when someone puts them in front of you (this is most undergrad STEM classes, including this one)
- Another (arguably equally important) skill: figuring out where you are in a particular era, and what tools and data make certain problems more ripe for solving than others
- Science is a lot about trying to solve the right problem at the right time with the right people. Knowing where you are in that landscape is just as important as gaining technical skill.
- Models are built with data in mind. So in this grand scheme of “science” and solving the right problem, if you're working in deep learning, a lot of that means picking the right data source. For us, picking which data we think about is as important as everything that comes afterward.

Why focus so much on the data source?

- Whether or not you're planning to do research, you'll probably want to have your finger on the pulse of what's going in a promising direction.
 - For us, I'm delving into “why language” because that's relevant to today's models.
 - For you, think about what problem your lab/company is trying to solve. Do you agree with it? Does it seem like a promising data source with close-to-answerable open questions?
- More on this : Stanford [CS 197: Computer Science Research](#); see “Vectoring in Research”

Presupposition

1 Characterizing Presupposition

1.1 Introduction

Speakers take a lot for granted. That is, they *presuppose* information. As we wrote this, we presupposed that readers would understand English. We also presupposed as we wrote the last sentence, repeated in (1), that there was a time when we wrote it, for otherwise the fronted phrase “as we wrote this” would not have identified a time interval.

(1) As we wrote this, we presupposed that readers would understand English.

Further, we presupposed that the sentence was jointly authored, for otherwise “we” would not have referred. And we presupposed that readers would be able to identify the reference of “this”, i.e., the article itself. And we presupposed that there would be at least two readers, for otherwise the bare plural “readers” would have been inappropriate. And so on.

- Assumptions and assertions we decide on when we make an argument
- **What is a presupposition we make when a class decides to teach about about LLMs and transformers?**

Why did the recent GenAI boom start with language?

- There definitely was a lot of interesting work with other modalities (e.g. images) leading up to this
 - e.g. Generative Adversarial Networks (GANs) used to make deep fakes
- But none of them really took off in the way that models that used language did. Even the the GenAI image tools that became big around the same time as ChatGPT (e.g. diffusion-based image tools like DALL-E) still used language modeling as a core part of their pipeline (you have to give DALL-E a language-based prompt).
- So it looks like we can infer there might be some helpful properties of language

Why do you think language-based models became big? Think about it for a minute, and then chat with the person next to you.

How does knowledge come to be? We call this “epistemology”.

Epistemology

🌐 140 languages ▾

Article [Talk](#)

Read [Edit](#) [View history](#) ⋮

From Wikipedia, the free encyclopedia



"Theory of knowledge" redirects here. For other uses, see [Theory of knowledge \(disambiguation\)](#). "Epistemic" redirects here. For the alternative name for cognitive science, see [Epistemics](#). For the Keep of Kalessin album, see [Epistemology \(album\)](#). Not to be confused with [Epidemiology](#).

Epistemology is the branch of [philosophy](#) that examines the nature, origin, and limits of [knowledge](#). Also called the **theory of knowledge**, it explores different types of knowledge, such as [propositional knowledge](#) about facts, [practical knowledge](#) in the form of skills, and [knowledge by acquaintance](#) as a familiarity through experience. Epistemologists study the concepts of [belief](#), [truth](#), and [justification](#) to understand the nature of knowledge. To discover how knowledge arises, they investigate sources of justification, such as [perception](#), [introspection](#), [memory](#), [reason](#), and [testimony](#).

What do we know?

What is it possible to know?

How do you decide what “knowing” means?

Thinking about what we call “information” is helpful when we’re trying to decide how our models handle that information.

A language property: **dense, discrete units of meaning**

Language has dense, well-defined units of meaning (e.g. words, prefixes, suffixes).

- Remember what we said in CNNs about a single pixel not carrying a lot of information, and how we wanted to create a feature map so that we don't have to deal with every pixel?
- If you work with images, you are spending a lot of time/computation on figuring out (“learning”) what the unit of meaning should be. It's easy to get “distracted” (spend a lot of time/computation) on things like local texture patterns that may not buy you very much meaning.
- **Why do images have these possibly-not-worth-learning local properties, and language doesn't?** Hint: what if we specifically said photographs, and excluded drawn images? Chat with the person next to you.

A language property: **dense, discrete units of meaning**

Why do images have these possibly-not-worth-learning local properties, and language doesn't?

- A lot of cognition happened up front to give you what you see with language! Someone *chose* to describe whatever is written, whereas a photo captures *every detail*, whether or not it carries meaningful information.
- That means that you get a lot of free pre-processing if you choose language over photos; the writer took care of creating something like a “feature map” for you
- This might be a different story if we trained entirely on drawings—someone *chose* to draw certain parts and not others. But we now have to pick very specific corpuses of drawings and choose a level of abstraction...this isn't standardized the way that language is. (This is epistemologically interesting.)

A language property: model objectives align with meaningful capabilities

- “Dense, discrete units of meaning” (what we just talked about): images require a lot of resources to get a relatively small amount of meaning
- A sort of corollary to this point: doing a really good job of image prediction doesn’t necessarily mean that you’ve learned anything meaningful that people would want to know.
- Is correctly predicting every pixel in an image a capability that you care about?
- “What the neural network learns is some representation of the process that produced the text. This text is actually a projection of the world.” – Ilya Sutskever in “AI Opener: OpenAI’s Sutskever in Conversation With Jensen Huang”
 - The leap made here in the “projection of the world” is that it’s how humans write about the world. This is important because we all have agreed upon conventions (to leverage) and not-agreed upon biases (to be aware of)

A language property: the data is its own label

- “Self-supervised”
- No data labeling needed
- Also – can slide the window and create lots of examples with the same dataset
- Would this be true of videos? Treating each frame as an input, and future frames as labels for previous frames? Yes. But runs into the problems of images mentioned previously.

Welcome	to	deep	learning
t_1	t_2	t_3	t_4



A language property: **language as a universal task interface**

- You can use language to: summarize, translate, write code, critique an argument, describe an image
- With image-only inputs/outputs, the only thing you can do is generate images. Expressing the complex ideas needed for the task above then ends up being a language task.

Does this mean you should only work with language?

- No!
- It just means that as you are working with other modalities, keep in mind what affordances they may or may not have compared to language, and how your model is accounting for them.

Now that we have these properties of language...

1. What is our data and why? **Why language** ✓
2. How are we representing the data? **Tokens and embeddings**
3. What is the architecture of the model we're using? **Transformers**
4. How does this architecture work? **Attention**

How do we represent language? Why tokens?

- We've talked about "next word prediction" and mostly talked about "words" as the input unit
- Words are a good idea...but there are a few problems:
 - Words don't fit in a fixed dictionary
 - Word vocabularies get huge and waste slots
 - Splitting text into words is hard
- What is a token? A reference example:
 - The phrase "I am learning" might be split into three sub-word tokens: "I", " am", " learn", and "ing"

Words don't fit in a fixed dictionary

- A model's vocabulary must be fixed in advance: every entry gets an ID + embedding (we'll talk about embeddings later)
- ...but words are effectively infinite: new slang, names, typos, URLs, code, other languages
- Anything off the list becomes "unknown"

Tokens address this because:

- Sub-word pieces can spell any word, even ones that didn't exist at build time
- A few thousand fragments cover infinite words

Word vocabularies get huge and waste slots

- Real coverage needs hundreds of thousands to millions of entries
- Every entry costs an embedding to store and learn
- "cat / cats / Cat / CAT" become separate, unrelated IDs — model can't see they're related

Tokens address this because:

- Tokens keep vocabulary small (tens of thousands) and expose shared structure
- "cat" + "s" reuses the "cat" piece, so learning transfers

Splitting text into words is hard

- Assumes you can cleanly find word boundaries—you often can't
- Language dependent: Chinese/Japanese use no spaces; German glues compounds together
- Punctuation, contractions ("don't"), hyphens, and code break any rule

Tokens address this because:

- Tokenizers skip "what is a word" and learn useful chunks statistically (similar to what we saw the simple RNN do yesterday)
- Common words stay whole ("the"); rare words break into parts

Why not letters?

- Think about it, and chat with the person next to you.
- What do we think?

Tokens: A nice middle ground between words and letters

- Letters = too fine: sequences get very long, each carries little meaning
 - Remember what we said about individual pixels for CNNs? We don't want to be dealing with letters as individual input.
- Whole words = too coarse: infinite, oversized, brittle, blind to shared structure
- Sub-word tokens: small, fixed, learnable set that can spell anything
- Compact and exposes structure words share
 - “Learn” and “ing”--both of those actually do give you some meaning/structure

You now have some insight about the “strawberry” issue.

- Remember when LLMs couldn't count the r's in “strawberry”?
- ML models are really good at doing things with the units they're designed for as input. For LLMs, those units are tokens.
- Asking it to count the number of “r”s means asking it to open the black box that is a token. This is like asking a CNN to handle something below the pixel level—it can't.

How did this issue get fixed?

- Nowadays, whatever tool you're using probably just calls another tool/calculator to actually count.
- It can also do chain of thought: Instead of jumping straight to a number, the model writes: "s-t-r-a-w-b-e-r-r-y." Now each letter is its own token in the context.

Now that we have these properties of language...

1. What is our data and why? **Why language** ✓
2. How are we representing the data? **Tokens** ✓ and **embeddings**
3. What is the architecture of the model we're using? **Transformers**
4. How does this architecture work? **Attention**